

7 - Genetic Algorithms

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Represent candidate solutions as chromosomes.
- Implement selection, crossover, mutation, and fitness evaluation.
- Study the effect of population size and mutation rate.

Background

Genetic algorithms are population-based optimization methods inspired by evolution. They are useful when a differentiable objective is unavailable or the search space is discrete and large. Their performance depends strongly on representation and parameter choices.

Core Concepts

Concept	Meaning in this lab
Population	Set of candidate solutions maintained together.
Fitness	Numerical score used to rank candidates.
Selection	Biases reproduction toward fitter candidates.
Crossover	Combines information from two parents.
Mutation	Introduces random variation and preserves diversity.

Guided Procedure

1. Use a binary chromosome and define a simple objective such as maximizing the number of 1s or matching a target bit string.
2. Create a random initial population.
3. Evaluate fitness for every chromosome.
4. Select parents, apply one-point crossover, and mutate individual bits with a small probability.
5. Keep the best solution and repeat for a fixed number of generations.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
import random

def fitness(bits, target):
    return sum(a == b for a, b in zip(bits, target))

def mutate(bits, p=0.02):
    out = bits[:]
    for i in range(len(out)):
        if random.random() < p:
            out[i] = 1 - out[i]
    return out

def crossover(a, b):
    k = random.randrange(1, len(a))
    return a[:k] + b[k:]
```

Lab Exercises

Task 1. Implement roulette-wheel or tournament selection.

Task 2. Plot best fitness by generation.

Task 3. Compare mutation rates 0, 0.01, 0.05, and 0.20.

Task 4. Extension: encode an integer optimization problem rather than a bit-matching task.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why is mutation necessary even when crossover is used?
2. What happens if selection pressure is too strong?
3. Why is chromosome representation important?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.